NutriGent

An Agentic Diet and Allergy Assistant

An LLM agent kept away from the safety decision

Multi-Agent Systems, Project Report

Kiana Alian
kiana.alian@studio.unibo.it · Matriculation number 0001153711
October 2026

Abstract

NutriGent is a self-hosted agentic assistant for people who manage food allergies and a calorie goal. It checks whether a food is safe for the user's allergies, plans a week of meals, derives a pantry-aware shopping list, tracks calories and macronutrients and answers with images rather than long text. Its user interface is a Telegram bot. The system combines a large-language-model agent (Hermes Agent running GPT-5.5), which converses with the user and extracts ingredients from photos and dish names, with a deterministic engine written in Python. Its guiding principle is *the model proposes, deterministic code disposes.* The language model never decides whether a food is safe; an auditable rule-based safety gate, modelled as an artifact in the sense of the Agents & Artifacts meta-model, issues every verdict and fails closed. The report analyses the system through the concepts of the course (agent properties, BDI, agents and artifacts, LLM-based agents), describes its architecture, workflows and design choices, documents the defects found and fixed with a test-first process, and evaluates it with 68 automated checks and an 80-action end-to-end test against a simulated messaging API. It ends with the deployment, the limitations and a roadmap.

Work split. This is an individual project. Design, implementation, testing, deployment and this report are the work of Kiana Alian.

Contents

1 Introduction **2**

2 Background and Connection to the Course **2**
2.1 Agents and their properties . . . 2
2.2 The BDI model . . . 3
2.3 Agents and Artifacts . . . 4
2.4 LLM-based agents . . . 4
2.5 NutriGent as a multi-agent system . . . 5

3 Employed Technologies **5**

4 The System **6**
4.1 Project folder structure . . . 6
4.2 Architecture . . . 6
4.3 Core components . . . 7
4.4 User interface . . . 8

5 Design and Analysis **9**
5.1 The deterministic safety gate . . . 9
5.2 Routing policy, or who may answer what . . . 11
5.3 The agent bridge . . . 11
5.4 Sign-up as a finite-state machine . . . 12
5.5 Nutrition model . . . 12
5.6 Data model . . . 13

6 Workflows **13**
6.1 Life of a message . . . 13
6.2 Food safety check . . . 14
6.3 Weekly plan, tonight's meal and shopping . . . 14
6.4 Logging and tracking . . . 15

7 Development Process **15**
7.1 Method . . . 15
7.2 Defects found and fixed . . . 16

8 Results **17**
8.1 Automated tests . . . 17
8.2 End-to-end test against a simulated messaging API . . . 17
8.3 Agent integration on the server . . . 18
8.4 Feature coverage . . . 18

9 Deployment **18**

10 Privacy, Safety and Ethics **19**

11 Limitations and Future Work **19**

12 Conclusion **20**

13 Reproducibility **21**

A Algorithms **23**

B Feature Catalogue **25**

C Agent Skills and Planned Schedule **27**

1 Introduction

People with food allergies make many small safety decisions every day. They read labels, recognise hidden ingredients (whey and casein are milk, Worcestershire sauce contains anchovy, E1105 is egg lysozyme), plan meals that respect allergies, preferences, budget and time, and often also try to keep a healthy calorie balance. Existing diet applications tend to fail them in four ways. They are *reactive*, because nothing happens until the app is opened. They are *siloed*, because the barcode scanner does not know the meal plan and the calorie counter does not guard allergens. They are *unaware of the pantry*, and they are *unable to learn* from week to week.

General-purpose chatbots could help with planning and conversation, but a language model can produce confident wrong answers [1]. For a user with a severe allergy a wrong "safe" can mean anaphylaxis. NutriGent was therefore built around four goals.

G1 Safety first
: Never declare a food safe when the evidence is insufficient, that is, fail closed.

G2 Low friction
: Meet the user in an everyday messaging app, prefer tappable buttons to typed commands, answer with images.

G3 Personalisation
: Calorie and macro targets, weekly plans, shopping lists, progress charts, and an agent that remembers each user.

G4 Privacy
: Self-hosted, so health data stays in a local database on the operator's server.

The deliverables of the project are the agentic engine with its deterministic safety core, the integration of an LLM agent (Hermes Agent), a Telegram bot as the user interface, an automated test suite including an end-to-end test against a simulated messaging service, a hardened deployment on a Linux server, the documentation in the repository (`README.md`, `FEATURES.md`) and this report.

The report is organised as follows. Section 2 relates the system to the concepts of the course. Section 3 lists the technologies used. Section 4 describes the architecture and its components, and Section 5 analyses the main design choices, namely the safety gate, the routing policy, the agent bridge, the sign-up state machine, the nutrition model and the data model. Section 6 follows the main workflows end to end. Section 7 describes the development process and the defects it uncovered, and Section 8 evaluates the system. Sections 9 to 11 cover deployment, ethics, limitations and the roadmap, and Section 13 explains how to reproduce the results.

2 Background and Connection to the Course

NutriGent is an applied system rather than a simulation, but its design questions are the classic questions of multi-agent systems. Which entities should be autonomous and which should be passive tools? How do they interact? How does a system stay predictable when part of it is a non-deterministic reasoner? This section introduces the concepts used throughout the report.

2.1 Agents and their properties

Following Wooldridge and Jennings [2, 3], an *agent* is a computer system situated in an environment and capable of autonomous action in it to meet its design objectives. A *weak* notion of agency asks for four properties. *Autonomy* means that the agent acts without direct intervention and controls its own state. *Reactivity* means that it perceives its environment and responds in time. *Proactiveness* means that it takes the initiative to pursue goals. *Social ability* means

that it interacts with other agents, humans included. Russell and Norvig [4] add the distinction between simple reflex agents, model-based agents that keep internal state, and goal-based agents that choose actions in view of a goal.

This list is exactly what separates an agent from a chatbot, since a chatbot is reactive and social but neither autonomous nor proactive. Table 1 states how far NutriGent realises each property today. Reactivity, social ability and state are complete, while proactiveness is designed but not yet switched on.

Table 1. Weak-agency properties and their realisation in NutriGent.

Property	Realisation in NutriGent	Status
Reactivity	Every incoming message, button tap or photo is perceived by long polling and answered within seconds, deterministic actions in under one second and agent replies in 10 to 20 s.	working
Social ability	Natural-language conversation in the user's language through Hermes, structured interaction through buttons, one memory per user.	working
State (model-based)	Profile, pantry, meal schedule, shopping list, food, weight and symptom logs are persistent beliefs in SQLite, and the agent keeps a long-term memory.	working
Autonomy	The agent decides how to answer open questions and which of its safe tools to use, while the engine owns the safety decision.	partial
Proactiveness	Scheduled jobs (daily preview, Sunday plan and report, Saturday shopping reminder, pantry sync) are written as skills but not yet registered with the scheduler.	roadmap

2.2 The BDI model

The Belief-Desire-Intention (BDI) model [5], implemented for example by the AgentSpeak interpreter Jason [6], describes an agent by its *beliefs* (what it holds true about the world), its *desires* (states it would like to bring about) and its *intentions* (the desires it has committed to, together with the plans chosen to achieve them). In AgentSpeak a plan consists of a triggering event, a context and a body. When the event occurs and the context holds in the belief base, the body is executed.

NutriGent is not written in a BDI language, but the BDI vocabulary maps naturally onto it and was used as a design lens (Table 2). The most useful correspondence is between AgentSpeak plans and the 15 *skills* of the Hermes agent. Each skill is a Markdown procedure with a trigger ("a photo arrives", "Sunday at 9.00"), applicability conditions ("the user has a profile") and a body ("extract, call the gate, send the card"). The plan library is therefore explicit and inspectable, as in Jason, even though the interpreter is a language model rather than a reasoning cycle.

Table 2. BDI concepts and their counterparts in NutriGent.

BDI concept	Counterpart in NutriGent
Beliefs	Rows in SQLite (allergies with severity, targets, pantry, schedule, logs) and the agent's long-term memory of each user.
Desires	G1, keep the user safe from their allergens, and G3, keep weekly nutrition on target, with secondary desires such as using expiring pantry items first.
Intentions	The committed weekly plan (`meal_schedule`) and tonight's meal. A "too tired" message drops the intention for tonight and re-schedules it.
Plans and plan library	The 15 Hermes skills (trigger, context, body) and, for the deterministic part, one handler function per action.
Events	Incoming messages, button taps and photos and, on the roadmap, timer events.

2.3 Agents and Artifacts

The Agents & Artifacts (A&A) meta-model [7, 8] separates two first-class abstractions. *Agents* are autonomous and pro-active. *Artifacts* are passive, function-oriented entities that agents *use*. They expose *operations* that agents can invoke and *observable properties* that agents can perceive, and they behave the same way for every user. Artifacts model tools, shared resources and the environment in general, and environment programming means designing that environment explicitly rather than hiding it inside agents.

This distinction is the central design decision of NutriGent, where safety-critical functions are artifacts and not agents.

- The **safety gate** is an artifact with one operation, `safety_verdict(extraction, profile)`, whose result depends only on its inputs and on the synonym tables.
- The **nutrition calculator**, the **chart renderers** and the **database** are artifacts that every component uses in the same way.
- The **messaging interface** (the Telegram Bot API) is the environment boundary through which users are perceived and acted upon.

The language model, which is autonomous, is allowed to *propose* (extract ingredients, write a plan, chat) but must *use* the gate artifact for every safety decision. Because the artifact is deterministic, it can be tested exhaustively, and its behaviour does not change when the model is replaced or behaves unexpectedly.

2.4 LLM-based agents

Recent agents use a large language model as their reasoning core, extended with tools, memory and planning [9]. ReAct [10] interleaves reasoning traces with tool actions, and this is how Hermes Agent works. In each turn the model may call a tool (search the web, read a skill, query its memory) and then continue. Two known weaknesses shape the design of NutriGent. The first is *hallucination* [1]. The model can state falsehoods fluently, so it must not own decisions whose errors are dangerous. The second is *prompt injection* [11]. Text supplied by users or found on web pages can steer the model, so an agent exposed to arbitrary users must not hold dangerous tools. NutriGent answers the first with the gate artifact and the second by giving the agent only harmless toolsets (Section 5.3).

2.5 NutriGent as a multi-agent system

Viewed as a MAS (Figure 1), NutriGent has three kinds of agents and a set of artifacts. *Users* are human agents with their own goals. The *front-desk agent* (`app/bot.py`) is a reactive agent that perceives every incoming message and decides who handles it, and it is the only holder of the bot token. The *conversational agent* (Hermes) is a deliberative LLM agent with memory, used for open conversation and for estimates. Interaction happens through messages that correspond closely to FIPA communicative acts [12]. A user's "can I eat tiramisu?" is a *query-if*, a verdict card is an *inform*, a button that asks for a meal swap is a *request*, and the front-desk agent forwarding a message to Hermes is a delegation. Coordination is organisational and role-based, because the routing policy of Section 5.2 assigns each kind of message to the role that is allowed to answer it.

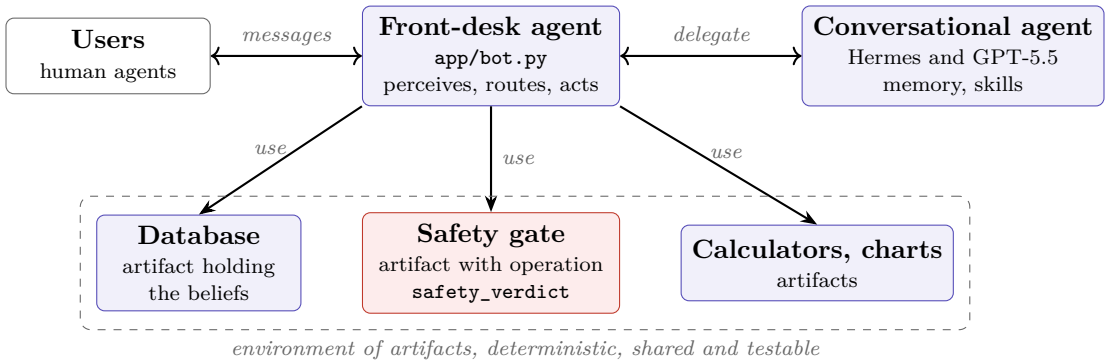


Figure 1. NutriGent seen as a multi-agent system. Agents decide and artifacts are used. The conversational agent never decides food safety and can only reach the user through the front desk.

3 Employed Technologies

Python 3.12. The whole engine is written in Python using mostly the standard library, with `urllib` for HTTP, `sqlite3` for storage, and `json`, `threading` and `subprocess`. Application code is about 1,900 lines and tests about 1,100 lines.

Telegram Bot API as the interface [13]. Telegram is only the user interface, and the engine does not depend on it. The bot uses *long polling* (`getUpdates`), so the server needs no public port, and three kinds of interface elements, a persistent reply keyboard under the input field, inline keyboards attached to messages, and the slash-command menu.

SQLite. One database file holds all state in 15 tables. Migrations are idempotent and run at start-up, so an existing database is upgraded in place.

Matplotlib. Verdict cards, dashboards and charts are rendered as PNG images with the non-interactive Agg backend and sent as photos.

Hermes Agent [14] and GPT-5.5. Hermes Agent is an open-source agent runtime with tools, persistent memory, skills and a scheduler. It runs on the server under its own profile (persona, memory and 947 messages of history restored from a backup) and uses OpenAI's GPT-5.5 model. The engine calls it as a command-line program in one-shot mode (Section 5.3). An optional OpenAI API key enables direct vision extraction and meal planning in `app/llm.py`. Without it, the system uses built-in sample answers so that every flow can run offline.

pip and make. Build automation uses pip. The file `pyproject.toml` declares the project and reads its dependencies from `requirements.txt`, so `pip install -e .` prepares an environment. A small `Makefile` wraps the common tasks (`install`, `test`, `e2e`, `run`, `report`).

systemd and Ubuntu. The pilot runs as a hardened systemd service on Ubuntu 24.04.

LaTeX. This report is written in LaTeX and built with Tectonic (`make report`).

4 The System

4.1 Project folder structure

Figure 2 shows the repository. Runtime code is split between `app/`, which adapts the engine to the messaging interface, and `shared/`, which holds the engine and its artifacts and is independent of the interface.

```
nutrigent/
├── app/
│   ├── bot.py ..... polling loop and router, the front-desk agent
│   ├── onboarding.py ..... sign-up state machine
│   ├── handlers.py ..... one function per user action
│   ├── hermes_bridge.py ..... calls the Hermes agent for chat and estimates
│   ├── pending.py ..... what the user's next typed message means
│   └── llm.py ..... OpenAI adapter for extraction and planning
├── shared/
│   ├── safety_gate.py ..... deterministic verdict engine, an artifact
│   ├── nutrition.py ..... Mifflin-St Jeor targets and macro split
│   ├── visuals.py, charts.py, reporting.py ..... PNG cards and charts
│   ├── telegram_io.py, telegram_menu.py ..... interface client and keyboards
│   ├── db.py, schema.sql, allergen_seed.sql ..... storage, schema, 145 synonyms
│   └── prompts/ ..... extraction and planning prompts
├── skills/ ..... 15 Hermes skills, one SKILL.md each
├── tests/ ..... unit suites, end-to-end test, fixtures
├── deploy/ ..... systemd unit and Hermes wrapper
├── report/ ..... this report with its figures
├── AGENT_PROMPT.md ..... persona and rules for the Hermes agent
├── README.md, FEATURES.md ..... documentation
├── pyproject.toml, requirements.txt, Makefile ..... build automation
└── .gitignore, .env.example ..... repository hygiene and configuration
```

Figure 2. The project folder structure.

4.2 Architecture

Figure 3 shows the deployed architecture. Users talk to the system through the Telegram interface. The module `app/bot.py` fetches incoming messages and routes each one. Commands, buttons and photos go to the deterministic handlers, and open conversation goes to Hermes through `app/hermes_bridge.py`. Handlers read and write the database, call the safety gate and the calculators, and send image cards. The language model appears in two places only, as the extraction step of a safety check (via `app/llm.py`) and as the conversational agent. Neither can issue a verdict.

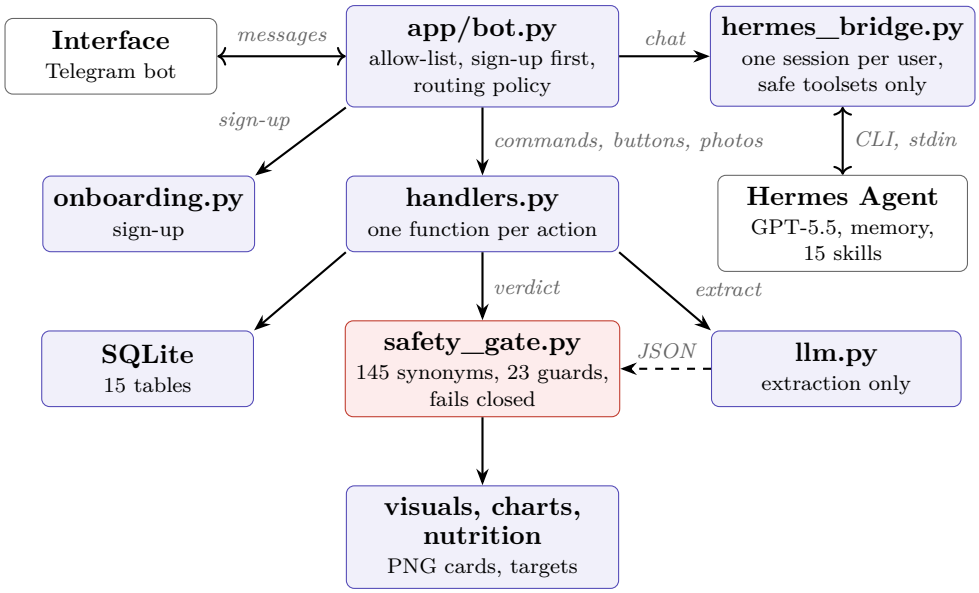


Figure 3. Deployed architecture. The red artifact decides safety, and the language model only proposes.

The architecture follows a layered style with dependency inversion at the edges. The `shared/` layer knows nothing about the messaging interface and can be tested in isolation, while `app/` adapts it to Telegram. The sending functions (`send_text`, `send_photo` and so on) are plain module attributes, so tests replace them with recorders and observe exactly what a user would receive. The same seam is used for the agent, since `hermes_bridge.RUNNER` can be replaced by a fake Hermes in tests. Replacing Telegram with another interface would only touch `app/`.

4.3 Core components

Front-desk agent (`app/bot.py`). An endless loop calls `getUpdates` with a long timeout and processes each incoming message. It applies the allow-list (`ALLOWED_TELEGRAM_IDS`), sends users who are in the middle of sign-up to the current question, translates button-grid labels such as “Today” into commands, and dispatches the message according to the routing policy of Section 5.2. Chat requests run in a small thread pool with two workers, so that a slow agent reply never blocks other users.

Sign-up (`app/onboarding.py`). A finite-state machine that interviews new users one question at a time and stores progress in the `onboarding_state` table (Section 5.4).

Handlers (`app/handlers.py`). One function per user action. The actions are check a food, show the dashboards, plan the week, show the shopping list, show tonight’s meal, cook, swap or show a recipe, log a meal or a symptom, the “too tired” fallback, the weekly report, the allergy card and the profile.

Pending input (`app/pending.py`). Some buttons ask the user to type something next (“Type a food name”, “Type it”, “Other symptom”). The expected kind of input is stored per user for 30 minutes, so the next plain message is routed to that action instead of the chat agent.

Agent bridge (`app/hermes_bridge.py`). Runs Hermes as a subprocess, keeps one agent session per user and returns either an answer or `None`, never an exception (Section 5.3).

LLM adapter (`app/llm.py`). The only module that calls the OpenAI API directly. It turns a photo or a dish name into a strict JSON *extraction* and produces weekly plans and meal estimates. When no key is configured it falls back to Hermes for estimates and to fixed sample answers otherwise.

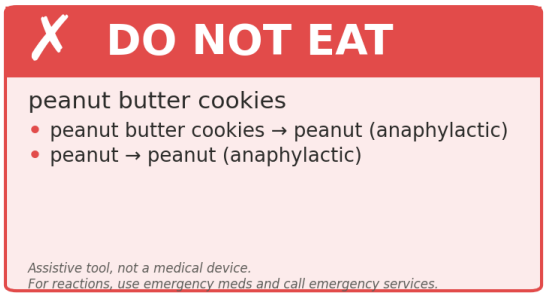
Safety gate (`shared/safety_gate.py`). The deterministic verdict engine (Section 5.1).

Calculators and renderers. `nutrition.py` computes targets (Section 5.5), `visuals.py` and `charts.py` render cards and charts, and `reporting.py` turns database rows into chart inputs.

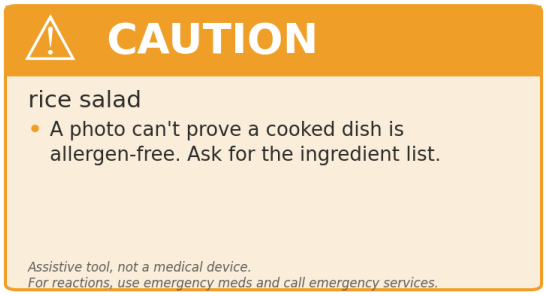
Hermes skills (`skills/`). Fifteen Markdown procedures that teach the agent the project's workflows (Table 15 in Appendix C).

4.4 User interface

Users never need to remember a command. Three interface layers coexist. The first is a persistent grid of 14 buttons under the input field, labelled without slashes ("Today", "Plan", "Check"). The second is inline buttons attached to messages, such as the 12-button main menu, sign-up choices, per-item shopping buttons, tonight's meal actions and the symptom picker. The third is the classic slash-command menu. Structured answers are images rendered from real database rows with one colour language, green for safe or on target, amber for caution or under target, red for unsafe or over target, purple for the brand and target lines, and grey for "cannot verify". Figures 4 and 5 were produced by the system during the end-to-end test, for test users with fictional data.



(a) Peanut butter cookies, anaphylactic peanut allergy.



(b) Photo of a cooked dish, where absence cannot be proven.

Figure 4. Verdict cards. Every DO NOT EAT and CAN'T VERIFY card carries emergency guidance.



(a) Today, with calorie ring and macro bars.

(b) Week, daily calories against the target.

(c) Month, 30 days with 7-day average and weight.

(d) The week's safety checks.

Figure 5. Tracking cards.

5 Design and Analysis

5.1 The deterministic safety gate

The gate turns an *extraction* and a *profile* into one of five verdicts, namely DO NOT EAT, CAUTION, LIKELY SAFE, SAFE and CAN'T VERIFY. The extraction is the only thing a language model contributes. It is a JSON object with the fields of Table 3.

Table 3. Fields of an extraction, with an example for a chocolate bar label.

Field	Meaning	Example
`image_type`	ingredient label, prepared food, menu, unreadable or not food	ingredient label
`readability`	how much of a label could be read, from 0 to 1	0.95
`identified_ingredients`	ingredients that are declared or visible	sugar, whey powder, cocoa
`likely_hidden_ingredients`	typical ingredients that are not visible	none
`precautionary_warnings`	"may contain" statements	may contain peanut

The gate works in four stages.

1. Resolving the user's allergies. Users write allergies in their own words. The gate maps them to the 14 canonical allergens of EU Regulation 1169/2011, Annex II [15], which also cover the nine major allergens of United States law [16]. They are celery, egg, fish, lupin, milk, mollusc, mustard, peanut, sesame, shellfish, soy, sulphite, tree nut and wheat. Aliases are resolved (*dairy* and *lactose* become milk, *gluten* becomes wheat, *seafood* becomes shellfish, mollusc and fish),

plurals are handled, and an ambiguous word maps to *every* allergen it could mean (*nuts* becomes tree nut and peanut). When two entries map to the same allergen the strictest severity wins. An allergen without a synonym list, for example kiwi, is matched by its literal name and can never yield SAFE. Algorithm 1 in Appendix A gives the procedure.

2. Matching. Each ingredient is normalised (lower case, punctuation removed) and compared with 145 synonyms, either as a substring ("whey" in "whey powder") or exactly (E-numbers such as E220 to E228). Table 4 shows a sample.

Table 4. Sample of the synonym table, which has 145 terms for 14 allergens.

Allergen	Examples of label terms that trigger it
milk	whey, casein, caseinate, lactose, lactalbumin, butter, E966, lactitol
egg	albumin, albumen, ovalbumin, lysozyme, mayonnaise, E1105
fish	anchovy, Worcestershire sauce, fish sauce, surimi, caviar
soy	lecithin, E322, tamari, miso, edamame, natto
wheat	gluten, semolina, spelt, durum, bulgur, seitan, farro, kamut, barley, rye, malt
sulphite	E220 to E228 (exact match), metabisulphite

3. False-positive guards. Substring matching alone would raise false alarms, so 23 guards skip known traps. Examples are coconut, almond, oat, rice and soy milk, cocoa and shea butter, peanut butter for milk, nutmeg, water chestnut and coconut for tree nuts, buckwheat for wheat, eggplant for egg, and "free from" claims.

4. Policy. The matches are classified by where they were found (declared ingredients, likely-hidden ingredients, precautionary warnings), and the policy of Table 5 is applied, strictest rule first.

Table 5. Verdict policy, checked from top to bottom.

#	Condition	Verdict
1	Input unusable, that is not food, unreadable, or a label with readability below 0.85	CAN'T VERIFY
2	A user allergen is among the declared ingredients	DO NOT EAT
3	A "may contain" trace, and the user's severity is severe or anaphylactic	DO NOT EAT
4	A trace or likely-hidden allergen at lower severity	CAUTION
5	The user has an allergen without a synonym list	CAUTION
6	Photo of a cooked dish or of a menu, and a severe or anaphylactic allergy	CAUTION
7	Photo of a cooked dish or of a menu otherwise	LIKELY SAFE
8	Complete, readable ingredient list with no match	SAFE

Properties. Three properties follow directly from this design and are checked by the tests. *Determinism* holds because the function reads only its inputs and static tables, so the same extraction and profile always give the same verdict. *Fail-closed* behaviour holds because every source of uncertainty (unusable input, unknown allergen, a photo that cannot show the recipe, a menu) maps to a verdict weaker than SAFE. *Monotonicity in evidence* holds because the policy checks hits before it ever reaches a SAFE rule, so adding an ingredient can only keep or lower the verdict.

A worked example. A user with a moderate fish allergy asks about a beef stew whose label lists "beef, Worcestershire sauce". The gate normalises the terms, finds the synonym "worcestershire" under fish, checks that no guard applies, and returns DO NOT EAT. The reason shown to the user names the matched term (Worcestershire sauce), the allergen (fish), the severity (moderate) and where it was found (the ingredients).

5.2 Routing policy, or who may answer what

Because one front-desk agent receives everything, the routing policy is where safety is enforced at the organisational level. The policy is an ordered list of rules (Algorithm 3 in Appendix A), summarised in Figure 6. The key rule is that *questions about food safety never reach the conversational agent.* A photo, a "can I eat …" question or a typed food name after the Check button always goes to the gate. The agent is also told, in the context of every message, never to judge food safety and to point the user to the Check button.

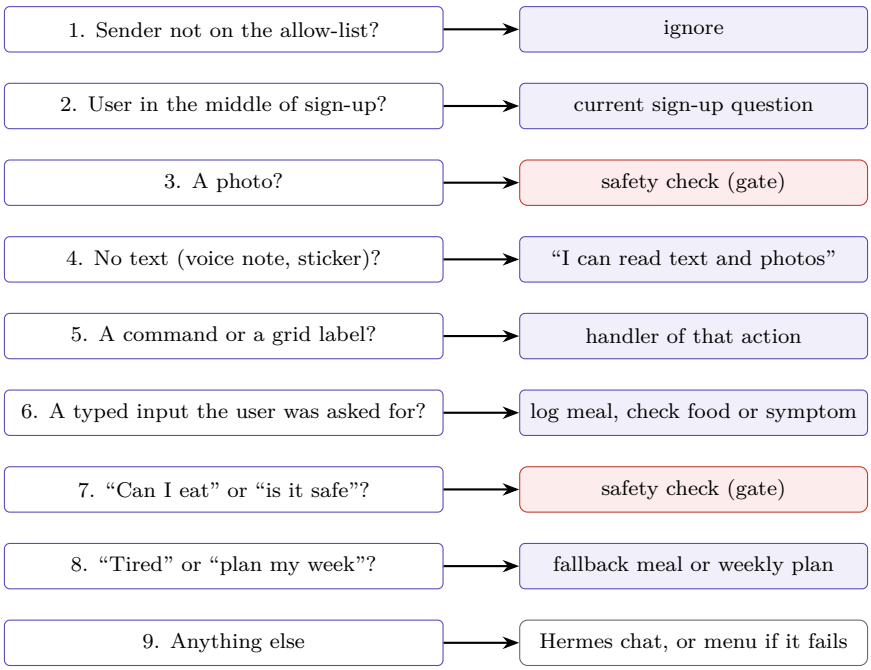


Figure 6. The routing policy of the front-desk agent, where the first matching rule wins. Red boxes mark the cases decided by the gate artifact.

5.3 The agent bridge

The bridge connects the front-desk agent to Hermes without giving up control. Its design responds to four concerns.

Isolation of user text. The user's message is passed on standard input (`--query-file -`), never through a shell or the argument list, so it cannot be interpreted as a command-line option or shell syntax.

Least privilege. Hermes is started with only five toolsets, `memory`, `session_search`, `skills`, `web` and `vision`. Terminal, file, code execution, browser and computer-use tools are not available, so a prompt-injection attempt [11] by a user cannot make the agent run commands on the server. A test on the server confirmed that the agent refuses shell requests. The number of reasoning turns is capped at six.

Per-user memory. Each user has one Hermes session, stored in the `hermes_sessions` table and resumed on every message, so the conversation has continuity. If a stored session cannot be resumed, the bridge forgets it and starts a fresh one (Algorithm 4). Every message carries a short context with the user's profile (allergies with severity, diet, goal) and three rules. The agent must reply in the user's language and in plain text, must never judge food safety, and must remember that it is not a medical device.

Graceful degradation. A missing binary, a non-zero exit code, a timeout (120 s for chat, 60 s for estimates) or an empty answer all yield `None`. The front desk then replies "Sorry, I couldn't answer that right now" with the main menu, so the user is never left in silence. While the agent thinks, the bot repeats the "typing" indicator every four seconds.

The same bridge provides *meal estimates*. When the user logs "chicken salad with rice", Hermes is asked for a JSON object with kcal and macros. The answer is *validated* before use (kcal strictly between 0 and 5000, each macro between 0 and 500 g), and otherwise the meal is stored without calories and the user is told. This is again *propose and dispose*, since the model suggests a number and code decides whether it is plausible.

5.4 Sign-up as a finite-state machine

The first version of sign-up had no memory of which question had been asked, which produced a loop in pilot testing (Section 7.2). It was redesigned as an explicit state machine (Figure 7). The state, made of the current step and the answers so far, is stored in the database before the next question is sent, so a restart never loses progress and an answered question is never asked again. Each step validates its input. Allergy taps toggle a selection and typed allergies are comma-separated. Numbers are checked against human ranges (age 13 to 100, height 120 to 230 cm, weight 30 to 300 kg), and invalid input re-asks the same question. Commands and grid buttons pressed during sign-up re-ask the current question instead of starting another action, while `/start` always restarts.

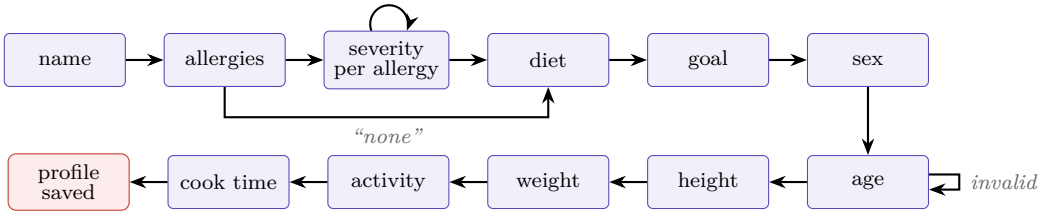


Figure 7. The sign-up state machine with eleven questions, mostly answered with buttons. Severity is asked once per allergy, "none" skips it, and invalid numbers stay in the same state.

5.5 Nutrition model

Daily energy targets use the Mifflin-St Jeor equation [17] for basal metabolic rate (BMR), multiplied by an activity factor to obtain total daily energy expenditure (TDEE), then adjusted for the goal.

$$\mathrm{BMR} = 10\,w + 6.25\,h - 5\,a + s, \qquad s = \begin{cases} +5 & \text{men} \\ -161 & \text{women} \end{cases} \tag{1}$$

$$\mathrm{TDEE} = \mathrm{BMR} \cdot f, \qquad f \in \{1.2,\ 1.375,\ 1.55,\ 1.725,\ 1.9\} \tag{2}$$

$$\mathrm{target} = \mathrm{TDEE} \cdot g, \qquad g = 0.80\ \text{(lose)},\ 1.00\ \text{(maintain)},\ 1.12\ \text{(gain)} \tag{3}$$

Here w is weight in kg, h height in cm and a age in years, and the five factors correspond to sedentary, light, moderate, active and athlete. Macronutrients are split into 30 % protein, 40 %

carbohydrate and 30 % fat, at 4, 4 and 9 kcal per gram. For example, a 29-year-old woman, 165 cm and 60 kg, moderately active and wanting to lose weight, gets a BMR of 1,325, a TDEE of 2,054 and a target of 1,643 kcal (123 g protein, 164 g carbohydrate, 55 g fat). Both this and a second example appear in the automated tests. The 20 % deficit and 12 % surplus are conservative defaults.

5.6 Data model

All state lives in one SQLite file, which keeps deployment simple and data on the operator's server (Table 6).

Table 6. Database schema (`shared/schema.sql`).

Table	Purpose
users	Interface identity, name, time zone, locale
dietary_profiles	Allergies with severity, diet, goal, calorie and macro targets, cooking time, budget, eating window, health flags
onboarding_state	Current sign-up step and answers so far
hermes_sessions	The agent session of each user
pending_input	What the user's next typed message means, with a 30-minute expiry
allergen_synonyms	145 label terms mapped to 14 canonical allergens (contains or exact)
synonym_exceptions	23 false-positive guards
pantry	Ingredients at home, quantity, unit, expiry date
recipes	Generated or imported recipes with macros and the gate's screening result
meal_schedule	Planned meals per day and their status (planned, cooked, skipped, swapped)
shopping_list_items	Weekly list with missing quantity, store section and status
food_log	Everything eaten or checked, with source, verdict, flagged allergens and macros
preference_events	Accepted, swapped, skipped, loved or hated recipes
weight_log, symptom_log	Weight over time, and symptoms with severity

6 Workflows

6.1 Life of a message

Figure 8 follows a free-text message and a safety question through the system. The two paths differ in who is allowed to answer, the agent for conversation and the gate for safety.

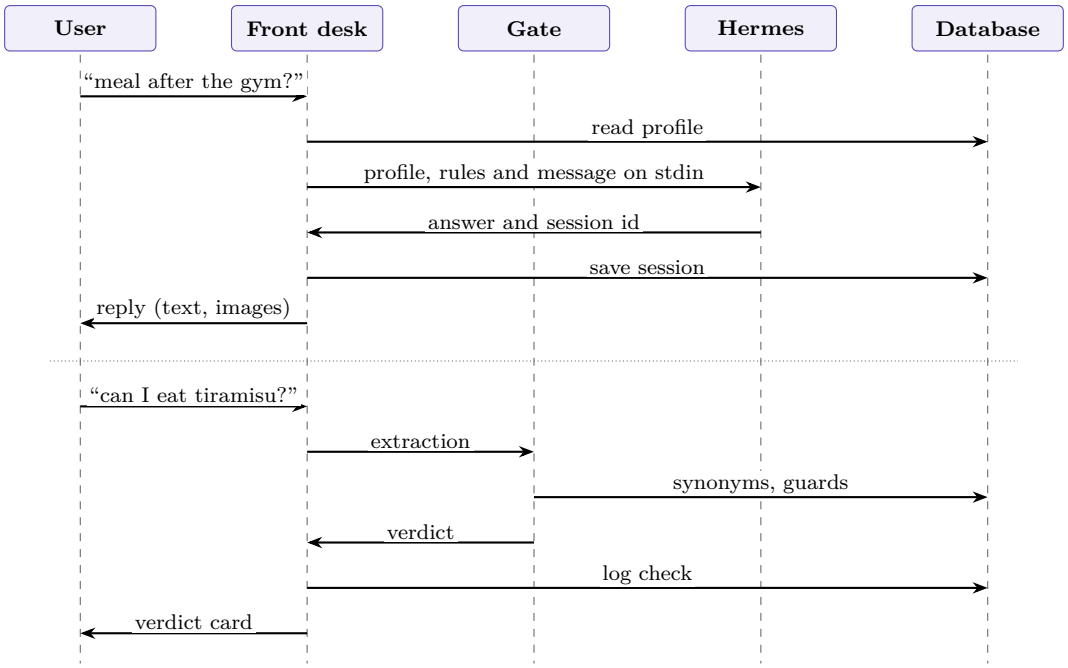


Figure 8. Two messages through the system. In the upper part, open conversation goes to the agent with the user's profile as context. In the lower part, a safety question never reaches the agent, and the extraction is the JSON proposed by the language model.

6.2 Food safety check

The user sends a photo, taps Check and types a dish name, or simply asks “can I eat …?”. The input is turned into an extraction. Photos go through vision, and for a dish name the model lists typical and likely-hidden ingredients (for tiramisu these are mascarpone, eggs, ladyfingers, cream and coffee). The gate returns a verdict, the system sends the verdict card with the textual reasons, and the check is logged in `food_log`, which feeds the weekly safety chart. The “Scan a menu” button routes the next photo through the same path, and a menu, like a photo of a cooked dish, can never be called SAFE.

6.3 Weekly plan, tonight's meal and shopping

The plan action sends the profile and pantry to the planner model, which returns seven days of recipes. The plan is *not trusted*, so every recipe is re-screened by the gate before it enters the schedule. Because a recipe's ingredient list is complete, it is screened like a label, and a recipe that hits an allergen is marked “needs regeneration”. The shopping list is computed as *ingredients required minus pantry stock*, with quantities, grouped by store section. “Tonight” shows the planned meal with three actions. *Cooking it* marks it cooked and logs its macros, *Swap* exchanges it with the next planned meal, and *Show recipe* sends ingredients and steps. Each shopping item has “bought” and “not available” buttons, and the system checks that the item belongs to the user who taps it.



(a) Plan card for a user with an anaphylactic peanut allergy.



(b) Shopping list grouped by store section.



(c) Profile card created by the sign-up.

Figure 9. Planning, shopping and profile cards.

6.4 Logging and tracking

"Log" offers three ways to record a meal. "Type it" waits for a description, asks the agent for an estimate (validated as described in Section 5.3) and stores a `food_log` row. Photo and voice logging are on the roadmap and say so. "Symptom" offers a picker (bloating, low energy, headache, skin, other) and stores the symptom with a timestamp, the raw material for the planned food-symptom correlation. "Today", "Week", "Month" and "Report" render the cards of Figure 5 from the logs.

7 Development Process

7.1 Method

Development followed a *test-first* discipline [18]. Every behaviour change started with an automated test, which was run and seen to fail for the expected reason before the code was changed, and the whole suite was run again before each deployment. A *model-first* step preceded the code wherever the behaviour was non-trivial. The verdict policy was written as a table (Table 5), the sign-up as a state machine (Figure 7) and the routing as an ordered rule list (Figure 6) before being implemented.

The repository follows a simplified GitLab flow, in which `main` is always deployable and each change is a focused commit made only after the suites pass locally. The release procedure is to run all suites locally, copy the committed tree to the server, run all suites on the server, restart the service and read the logs. The repository is kept clean by a `.gitignore` that excludes secrets (`.env`), databases (which contain health data), caches, virtual environments, build output, LaTeX output, IDE configuration and operating-system files. No binaries other than the report figures and one test photo are tracked.

7.2 Defects found and fixed

Testing against realistic inputs uncovered several defects, some of them safety-relevant. Table 7 lists them, and each fix is protected by a regression test.

Table 7. Defects found during the project. All are covered by regression tests.

Defect	Effect	Fix
Fail-open allergy names	A user who wrote “dairy”, “peanuts”, “gluten” or “nuts” matched nothing, so every food was SAFE, a direct violation of G1.	Alias and plural resolution, ambiguous words mapped to all candidates, unknown allergens matched literally and capped at CAUTION.
Sign-up loop	Every message from a new user produced the same welcome and “what should I call you?”.	Sign-up rebuilt as a persisted state machine.
Menu photo called SAFE	A menu without a visible allergen word returned SAFE.	Menus treated like cooked dishes (policy rules 6 and 7).
Logging marked dinner cooked	Logging any snack marked tonight’s planned meal as cooked.	Logging no longer touches the schedule, only “Cooking it” does.
Rejected replies	“mac & cheese” made the interface reject the reply because of HTML parsing.	Messages are sent as plain text.
Every planned meal flagged	Recipes were screened as photos of cooked dishes, so for severe allergies every meal was “needs regeneration”.	Recipes screened as complete ingredient lists.
Text off the card	Long verdict reasons ran past the card edge.	Reasons wrap without splitting words.
13 silent buttons	Check, log, symptom and tonight sub-buttons had no handler.	Every button wired, typed follow-ups routed by `pending.py`.
Open agent gateway	On the previous server the agent, which can run shell commands, accepted any user and ran as root.	Unprivileged user, agent gateway off, allow-list, safe toolsets only.

The fail-open defect deserves a comment because it illustrates the value of the artifact design. The defect was in the gate, not in the model. Because the gate is deterministic, a table of before and after cases (Table 8) could be turned directly into tests, and the fix could be proven complete for every listed case.

Table 8. Behaviour of the gate before and after the fail-open fix. Each row is a test.

User wrote	Label contains	Before	After
dairy	milk powder	SAFE	DO NOT EAT
lactose	whey	SAFE	DO NOT EAT
Peanuts	peanut butter	SAFE	DO NOT EAT
gluten	wheat flour	SAFE	DO NOT EAT
nuts	almonds, or peanut oil	SAFE	DO NOT EAT
kiwi (no synonym list)	kiwi fruit, or water and salt	SAFE, SAFE	DO NOT EAT, CAUTION
egg	eggplant	DO NOT EAT (false alarm)	SAFE

8 Results

8.1 Automated tests

Table 9 lists the suites, and `make test` runs them all. All 45 unit tests and 23 safety fixtures pass (68 checks), on macOS and on the server.

Table 9. Automated test suites.

Suite	What it checks	Result
test_safety_gate	Direct and hidden allergens, traces by severity, unreadable labels, cooked dishes and menus, false-positive guards, E-numbers, typed allergy names, unknown allergens	23 of 23
test_chat_and_buttons	Free chat reaches the agent with the profile, food questions stay with the gate, agent failure falls back to the menu, every button, plan screening, card text wrapping, plain text	19 of 19
test_hermes_bridge	Output parsing, per-user sessions, stale-session retry, timeouts, missing binary, estimate validation (with a fake Hermes)	9 of 9
test_onboarding_flow	Full sign-up, no loop, bad numbers re-asked, no-allergy path, commands during sign-up, returning user	6 of 6
test_reply_keyboard	Grid labels without slashes, persistence, every label runs its command	5 of 5
test_access	Allow-list and ownership of shopping items	4 of 4
test_onboarding_buttons	Sign-up button layouts and callback formats	2 of 2
test_bot_flow	Simulated messages through router, handlers and image rendering	runs

8.2 End-to-end test against a simulated messaging API

Unit tests replace the network with recorders, so they cannot show what a user actually experiences. A fake Telegram Bot API was therefore written (`tests/e2e_fake_telegram.py`). It is a small HTTP server on the local machine implementing `getUpdates`, `sendMessage`, `sendPhoto`, `sendChatAction`, `getFile` and `answerCallbackQuery`. Like the real service, it rejects messages that break the HTML rules and photos that are not valid images. The *unmodified* polling loop of `bot.py` is pointed at it and driven through 80 scripted actions. They include 19 commands (one deliberately unknown), 29 button taps, two photos, eight free-text messages including Persian and the characters "&" and "<", a voice note, and the complete 21-step session of a brand-new user. The run takes about 20 seconds and peaks at about 120 MB of memory.

Table 10 compares the run before and after the work on the core flows. The final run gets a correct reply to every action, with identical results on the laptop and on the server.

Table 10. End-to-end results, before and after the core-flows phase.

Outcome	Before	After
Correct reply	66	80
No reply (buttons without a handler)	13	0
Rejected by the interface ("&" in a reply)	1	0
Total actions	80	80

A paraphrased excerpt of the new user's session shows the sign-up behaving as designed, including a grid button pressed during sign-up and an invalid age (Table 11).

Table 11. Paraphrased excerpt of the sign-up session in the end-to-end run.

User sends	NutriGent replies
/start	Greets the user as their diet and allergy assistant and asks what to call them
Nima	"Nice to meet you, Nima!" and asks about food allergies
[taps Milk], types "kiwi"	Shows the selection, first milk, then milk and kiwi, and offers Done
[taps the Today grid button]	"Let's finish setting you up first." and repeats the question
[Done], [Severe], [Mild]	Asks the severity of each allergy, then the diet
abc (as age)	"Please send a number between 13 and 100." and repeats the question
… [15 min]	Sends the profile card and says the user is set
/today	Sends the dashboard with 0 of 3068 kcal
/start	"Welcome back, Nima!"
check milk chocolate	DO NOT EAT, because milk chocolate contains milk, a severe allergy of this user

8.3 Agent integration on the server

The real agent was tested on the server inside a sandbox with the same restrictions as the service. A chat request returned a correct answer in about 10 seconds. A meal estimate for "chicken salad with rice" returned 650 kcal, 45 g protein, 65 g carbohydrate and 22 g fat in about 11 seconds, which passed the plausibility check. Asked to run a shell command, the agent refused, as it has no terminal tool. The system itself uses about 48 MB of memory at rest.

8.4 Feature coverage

NutriGent was designed as 48 features in eight areas (safety, planning, tracking, shopping and pantry, autonomous actions, visual replies, adaptations, integrations). Appendix B lists them with their status. Nineteen are working, 8 partial and 21 on the roadmap, and 15 further features were added during the project, all working. The core safety, tracking and visual features are complete. Most of what remains is autonomy (scheduled jobs), richer inputs (barcode, photo and voice logging) and adaptation.

9 Deployment

The pilot runs on an Ubuntu 24.04 virtual server with one vCPU and 2 GB of RAM, shared with an unrelated project. Because the interface is reached by long polling, no inbound port is opened. Table 12 summarises the configuration (`deploy/nutrigent-bot.service` in the repository).

Table 12. Deployment configuration.

Aspect	Choice
Isolation	Dedicated unprivileged system user, with code, virtual environment, data and agent profile under `/opt/nutrigent`
Process management	systemd service that starts on boot and restarts on failure, memory capped at 900 MB to leave room for agent calls
Hardening	`NoNewPrivileges`, `ProtectSystem=strict`, `PrivateTmp` and `ProtectHome`, with write access only to the data directory and the agent profile
Secrets	Bot token and allow-list in an `.env` file readable only by the service user and entered by the operator
Agent	Hermes Agent 0.21.6 under the same user, profile restored from backup, its own messaging gateway disabled so that the front desk is the only reader of the token, and called by the bridge with safe toolsets only
Database upgrades	Idempotent migrations at start-up that preserve user data
Resources	About 48 MB at rest for the system, and about 0.7 GB of 2 GB in use on the whole server

10 Privacy, Safety and Ethics

Not a medical device. NutriGent is an assistive tool. It does not diagnose or treat conditions, replace a dietitian, allergist or doctor, or guarantee allergy safety, but it reduces risk. This is stated at sign-up and on every DO NOT EAT and CAN'T VERIFY card, together with the advice to use emergency medication and call emergency services in case of a reaction.

Uncertainty is risk. Unreadable labels, unknown allergens, cooked dishes and menus never receive a plain SAFE for severe allergies. The asymmetry is deliberate, because a false "unsafe" costs a meal while a false "safe" can cost a life.

Responsibility. Keeping the decision in a deterministic artifact makes responsibility traceable. Every verdict comes with the exact term, allergen, severity and location that caused it, and the rule that produced it can be found in a table. A decision made inside a language model could not be explained or audited in the same way.

Data. Profiles, logs, plans and agent memory stay on the operator's server. For a safety check the photo or dish name and the allergen names are sent to the model provider for extraction, but the user's name, identity and history are not. Open conversation is processed by the model through the agent. Access is restricted by an allow-list, services run without root privileges, and secrets and health data are excluded from version control.

11 Limitations and Future Work

Limitations.

- **Proactiveness is not active yet.** The scheduled jobs exist as skills but are not registered, so the system is still reactive in practice.
- **Inputs.** Barcode lookup, photo logging and voice logging answer "coming soon".
- **Synonym coverage.** The 145 terms are a starting set that needs expansion and review by an allergy professional. The extraction step itself can miss an ingredient in a blurred photo, which the readability floor mitigates but cannot eliminate.

- **Model dependence.** Without an OpenAI key, photo reading and planning use sample answers, and without Hermes, chat falls back to the menu.
- **Static fallback.** The "too tired" suggestion is fixed text rather than chosen from the pantry.
- **Scale.** SQLite and long polling suit a pilot with 3 to 10 users, and larger use needs a client-server database and webhooks.

Roadmap. Table 13 gives the planned phases. The next architectural step is to make every action return a *reply object* (text, images, buttons) instead of sending it, and to expose the actions through a command-line interface. The agent's skills will then call one tested action each instead of improvising, so that the agent's actions inherit the reliability of the engine. In A&A terms, the agent will act on the environment only through artifacts with well-defined operations. The same step makes it straightforward to add other interfaces next to Telegram.

Table 13. Roadmap.

Phase	Scope	Status
1. Safety core and pilot	Gate, synonyms and guards, cards and charts, sign-up, button grid, access control, tests, deployment, agent restored	done
2. Core flows	Agent chat, every button, meal estimates, plain text, Tonight, swap, recipes screened as labels, with reply objects, CLI and a pantry-based fallback still to do	in progress
3. Better inputs	Barcode via Open Food Facts [19], per-dish menu verdicts, photo and voice logging	planned
4. Autonomy	Scheduled jobs for the daily preview and recipe, Sunday plan and report, Saturday shopping reminder, pantry sync and preference memory	planned
5. Insights and adaptation	Food-symptom correlation, weekly rebalancing, weight-based recalibration, profile editing, Persian interface	planned
6. Integrations and scale	Calendar, weather, to-do, notes, e-mail, smart scale, health flags, PostgreSQL and webhooks	planned

12 Conclusion

NutriGent shows that an LLM agent can be useful for everyday diet and allergy management when it is kept away from the decisions whose errors are dangerous. Separating *proposal*, where the agent extracts, plans, estimates and converses, from *disposal*, where deterministic artifacts decide, is the Agents & Artifacts distinction applied to a safety problem. It is also what made the system testable. Because the gate is a deterministic artifact, a serious fail-open defect could be found, expressed as a table of cases, fixed and proven fixed. The same principle guided the integration of the agent, which receives a restricted set of tools, one session per user, and a front-desk agent that decides which messages it may answer.

The pilot is deployed and working. Users sign up step by step, check foods by photo or name, chat with the agent in their own language, receive dashboards, plans and shopping lists, and every button answers. The main gap with respect to the agent properties of Section 2 is proactiveness. Switching on the scheduled jobs, on top of tested actions, is the step that will turn NutriGent from a responsive assistant into an agent that acts on its own initiative.

13 Reproducibility

The project is available at github.com/KianaAlian/NutriGent. Python 3.11 or newer is required, and `make` and, for the report, `tectonic` are optional conveniences. After cloning the repository into a folder named `nutrigent` and entering it, the following commands prepare and verify the system.

```
make install    # python3 -m venv .venv and pip install -e .
make test       # every unit suite
make e2e        # 80 actions against a simulated messaging API
make report     # builds report/main.pdf
```

To run the system with its Telegram interface, create a bot with @BotFather, copy `.env.example` to `.env`, set `TELEGRAM_TOKEN` (and optionally `OPENAI_API_KEY`, `ALLOWED_TELEGRAM_IDS` and `HERMES_BIN`), then run `make run` and send `/start` to the bot. Without an API key and without Hermes, every flow still works with sample extractions and plans, and free chat answers with the menu. The tests and the end-to-end run need no network access and no keys.

References

[1] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), article 248, 2023.

[2] Michael Wooldridge and Nicholas R. Jennings. Intelligent agents. Theory and practice. *The Knowledge Engineering Review*, 10(2), pp. 115-152, 1995.

[3] Michael Wooldridge. *An Introduction to MultiAgent Systems*. 2nd edition. John Wiley & Sons, Chichester, UK, 2009.

[4] Stuart Russell and Peter Norvig. *Artificial Intelligence. A Modern Approach*. 4th edition. Pearson, 2020.

[5] Anand S. Rao and Michael P. Georgeff. BDI agents. From theory to practice. In *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS-95)*, pp. 312-319, 1995.

[6] Rafael H. Bordini, Jomi Fred Hübner and Michael Wooldridge. *Programming Multi-Agent Systems in AgentSpeak Using Jason*. John Wiley & Sons, Chichester, UK, 2007.

[7] Andrea Omicini, Alessandro Ricci and Mirko Viroli. Artifacts in the A&A meta-model for multi-agent systems. *Autonomous Agents and Multi-Agent Systems*, 17(3), pp. 432-456, 2008.

[8] Alessandro Ricci, Michele Piunti and Mirko Viroli. Environment programming in multi-agent systems. An artifact-based perspective. *Autonomous Agents and Multi-Agent Systems*, 23(2), pp. 158-192, 2011.

[9] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), article 186345, 2024.

[10] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan and Yuan Cao. ReAct. Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[11] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz and Mario Fritz. Not what you've signed up for. Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*, pp. 79-90, 2023.

[12] Foundation for Intelligent Physical Agents. *FIPA Communicative Act Library Specification*. Standard SC00037J, 2002.

[13] Telegram. *Telegram Bot API*. core.telegram.org/bots/api, accessed October 2026.

[14] Nous Research. *Hermes Agent*, version 0.21.6. github.com/NousResearch/hermes-agent, accessed October 2026.

[15] European Parliament and Council of the European Union. Regulation (EU) No 1169/2011 on the provision of food information to consumers, Annex II, substances or products causing allergies or intolerances. *Official Journal of the European Union*, L 304, 2011.

[16] United States Congress. *Food Allergy Safety, Treatment, Education, and Research (FASTER) Act of 2021*. Public Law 117-11, 2021.

[17] M. D. Mifflin, S. T. St Jeor, L. A. Hill, B. J. Scott, S. A. Daugherty and Y. O. Koh. A new predictive equation for resting energy expenditure in healthy individuals. *The American Journal of Clinical Nutrition*, 51(2), pp. 241-247, 1990.

[18] Kent Beck. *Test-Driven Development by Example*. Addison-Wesley, 2002.

[19] Open Food Facts. *The free food products database*. world.openfoodfacts.org, accessed October 2026.

A Algorithms

Algorithm 1. Resolving the user's allergies

Input profile allergies $P = \{(name_k, sev_k)\}$, canonical allergens K, alias table Al;
Output R, mapping each canonical allergen to its strictest severity, and U, the names without a synonym list;
$R \leftarrow \emptyset$; $U \leftarrow \emptyset$;
foreach $(name, sev) \in P$ **do**
    $n \leftarrow$ normalise($name$);
    if $n \in K$ **then**
        $C \leftarrow \{n\}$
    else if $n \in Al$ **then**
        $C \leftarrow Al[n]$; `// ambiguous words map to every candidate`
    else if *n ends in "s" and n without "s"* $\in K$ **then**
        $C \leftarrow \{n$ without "s"$\}$
    else
        $U \leftarrow U \cup \{n\}$; $C \leftarrow \{n\}$; `// matched literally`
    foreach $c \in C$ **do**
        if $c \notin R$ ***or*** rank(sev) $\geq$ rank($R[c]$) **then**
            $R[c] \leftarrow sev$
return R, U;

Algorithm 2. Safety verdict

Input extraction E, profile P, synonyms S, guards G;
Output a verdict with reasons and hits;
if $E.type \in \{unreadable, not_food\}$ ***or*** *($E.type = label$* ***and*** *$E.readability < 0.85$)* **then**
    return *CAN'T VERIFY*
$(R, U) \leftarrow$ RESOLVE(P);
add $(u, u, contains)$ to S for every $u \in U$;
$D \leftarrow$ MATCH($E.ingredients, R, S, G$); $H \leftarrow$ MATCH($E.hidden, R, S, G$); $T \leftarrow$ MATCH($E.warnings, R, S, G$);
if $D \neq \emptyset$ **then return** *DO NOT EAT*;
if *some $t \in T$ has severity severe or anaphylactic* **then return** *DO NOT EAT*;
if $T \cup H \neq \emptyset$ **then return** *CAUTION*;
if $U \neq \emptyset$ **then return** *CAUTION*;
if $E.type \in \{prepared_food, menu\}$ **then**
    if *some allergen in R is severe or anaphylactic* **then return** *CAUTION*;
    return *LIKELY SAFE*;
return *SAFE*;
Function `Match(`*terms, R, S, G*`)`
    foreach *$t \in$ terms and $(c, s, m) \in S$ with $c \in R$* **do**
        $n \leftarrow$ normalise(t);
        if *($m = exact$* ***and*** *$n = s$)* ***or*** *($m = contains$* ***and*** *s occurs in n)* **then**
            if *no guard $g \in G[c]$ occurs in n* **then** record hit $(c, t, R[c])$;

Algorithm 3. Routing an incoming message (front-desk agent)

Input message u from the interface;
if *u.sender* $\notin$ *allow-list* **then return**;
if *u is a button tap* **then**
    if *sign-up in progress* **then** ONBOARDING(u) or re-ask, then **return**;
    clear pending input, dispatch to the handler named by the callback data, **return**;
if *sign-up in progress* **then** ONBOARDING(u) (commands re-ask the current question), **return**;
if *u has a photo* **then** clear pending input, CHECKFOOD(u), **return**;
if *u has no text* **then** reply "I can read text and photos", **return**;
$t \leftarrow$ GRIDLABELTOCOMMAND(*u.text*);
if *t is a command* **then** clear pending input, dispatch, **return**;
if $k \leftarrow$ *TAKEPENDING*(*u.sender*) **then** run the handler of k with t, **return**;
if *t matches a safety question* **then** CHECKFOOD(t), **return**;
if *t matches "tired"* **then** TOOTIRED(), **return**;
if *t matches "plan my week"* **then** PLANWEEK(), **return**;
run CHAT(t) in the background pool;

Algorithm 4. Asking the agent (`hermes_bridge.ask`)

Input user id, message m, context c;
Output $\{text, images\}$ or NONE;
$q \leftarrow$ context c followed by message m ; `// passed on stdin, never as arguments`
$sid \leftarrow$ stored session of id;
$r \leftarrow$ RUN(hermes, safe toolsets, at most 6 turns, resume sid if any, stdin q, timeout);
if $r =$ *NONE* ***and*** $sid \neq$ *NONE* **then**
    forget sid; $r \leftarrow$ RUN(…, no resume); `// stale session, start fresh`
if $r =$ *NONE* ***or*** *r is empty* **then return** *NONE*;
store *r.session* for id;
return $\{r.text, r.images\}$;

Algorithm 5. Weekly plan with re-screening and shopping list

Input profile P, pantry Q;
$W \leftarrow$ GENERATEWEEK(P, Q) ; `// proposal from the model, not trusted`
$need \leftarrow \emptyset$;
foreach *day d and meal* $r \in W$ **do**
    $v \leftarrow$ SAFETYVERDICT(label with the ingredients of r, P);
    if $v \in \{$*DO NOT EAT, CAUTION*$\}$ **then** mark r "needs regeneration";
    store r and schedule it on d;
    foreach *ingredient* $(n, q, unit) \in r$ **do** $need[n, unit] \leftarrow need[n, unit] + q$;
foreach $(n, unit) \in need$ **do**
    $missing \leftarrow need[n, unit] - Q[n, unit]$;
    if $missing > 0$ **then** add $(n, missing, unit, \text{section}(n))$ to the shopping list;

B Feature Catalogue

Status is **W** for working (implemented and exercised by tests or the end-to-end run), **P** for partial and **R** for roadmap (designed, not built). The same catalogue, with more detail, is kept in `FEATURES.md`.

#	Feature	St.	Notes
Safety			
F1	Photo safety check	W	Photo, then extraction, then gate, then verdict card
F2	Barcode check (Open Food Facts)	R	Procedure in a skill, no code path yet
F3	Text safety check	W	Dish decomposed into typical and hidden ingredients
F4	Hidden-allergen synonyms	W	145 terms for 14 allergens
F5	False-positive guards	W	23 patterns
F6	Per-allergen severity policy	W	Severity decides how traces are treated
F7	Fail-closed on unreadable input	W	CAN'T VERIFY below readability 0.85
F8	Restaurant menu scan	P	Button wired and never SAFE, per-dish list in a skill
F9	Allergy card for travel	P	English card, local language planned
F10	Cross-contamination guard	W	"May contain" handled by severity
F11	Multi-language labels	P	Prompt translates, not validated on real labels
Planning and meals			
F12	Weekly meal plan	P	Seven dinners, re-screened and stored, macro constraints only in the prompt
F13	Swap a single meal	W	Swaps with the next planned meal
F14	Leftover chaining	P	Requested in the prompt, tag on the card
F15	Recipe import and make-safe	P	Agent skill only
F16	Symptom-food correlation	R	Symptoms stored, analysis not built
Tracking			
F17	Personal calorie target	W	Mifflin-St Jeor at sign-up
F18	Daily macro dashboard	W	Ring and bars from the food log
F19	Meal logging (text, photo, barcode, voice)	P	Text with validated estimates, others planned
F20	Weekly rebalancing	R	Not started
F21	Recalibration from weight	R	Weight stored and charted
Shopping and pantry			
F22	Pantry-aware shopping list	W	Needed minus pantry, sections, per-item buttons
F23	Pantry sync four times a week	R	Skill written, not scheduled
F24	Spoilage routing	R	Expiry dates stored
F25	Macro-preserving substitutions	R	Described in a skill
F26	Saturday shopping reminder	R	Planned schedule
Autonomous actions			
F27	Tonight's-meal preview at 16.00	R	Skill written, not scheduled
F28	Recipe delivery at 17.30	R	Same as F27
F29	"Too tired" fallback	P	Pauses tonight's meal, fixed suggestion

#	Feature	St.	Notes
Visual replies			
F30 to F37	Verdict card, today, week, month, safety donut, plan, shopping and profile cards	W	All rendered from real rows
Adaptations and integrations			
F38 to F42	Batch cooking, skill progression, eating window, health flags, medication flags	R	Profile fields exist for F39 to F41
F43 to F48	Calendar, weather, to-do, notes, e-mail, smart scale	R	Through agent integrations
Added during the project (all working)			
N1	Step-by-step sign-up	W	Persisted state machine
N2	Allergy names as people type them	W	Aliases and plurals, fixes fail-open
N3	Unknown allergens never SAFE	W	Literal match, CAUTION cap
N4	Eggplant is not egg	W	New guard, migrated
N5	Button grid without slashes	W	Labels and typed words both work
N6	Allow-list	W	`ALLOWED_TELEGRAM_IDS`
N7	Per-user data ownership	W	Users change only their own items
N8	End-to-end test	W	80 actions against a fake messaging API
N9	Hardened deployment, restored agent	W	Unprivileged service, agent memory restored
N10	Agent chat behind the front desk	W	Per-user sessions, safe tools, fallback
N11	Every button answers	W	Sub-menus, picker, tonight actions, shopping items
N12	Real “Tonight”	W	Cook, swap, recipe
N13	Plain-text replies	W	“&” and “<” no longer break replies
N14	Menu photos never SAFE	W	Treated like cooked dishes
N15	Recipes screened as labels	W	Only real hits are flagged

C Agent Skills and Planned Schedule

Table 15. The 15 Hermes skills, which form the agent's plan library.

Skill	Trigger	Body
onboard-user	/start, new user	Interview → profile → menu
food-safety-check	photo, "can I eat X?"	Extract → gate → verdict card
charts-dashboard	/today, /week, /month	Food log → chart → photo
weekly-plan	Sunday 9.00, "plan my week"	Profile and pantry → plan → re-screen → card
shopping-assistant	/shopping, Saturday 18.00	Pantry difference → grouped list → reminder
daily-nudge	16.00, 17.30, 21.30	Preview → recipe → follow-up
log-meal	/log, plate photo, voice	Estimate → food log → dashboard
handle-friction	"tired", "sold out"	Fallback meal or substitution, rebalance
pantry-sync	Mon, Wed, Fri, Sun 23.00	Reconcile → subtract → expiry flags → memory
weekly-report	Sunday 20.00	Aggregate → three charts → correlation
update-profile	"I'm also allergic to …"	Patch profile → re-screen the week
eating-out	menu photo, /card	Per-dish gate → safe, ask or avoid
import-recipe	recipe URL, "make safe"	Extract → screen → substitute → rescale
life-sync	calendar, weather, to-do, e-mail	Integrations
visual-output	every structured reply	Convention, answer with images